RARS Tutorial

In this simple tutorial we will teach you how to:

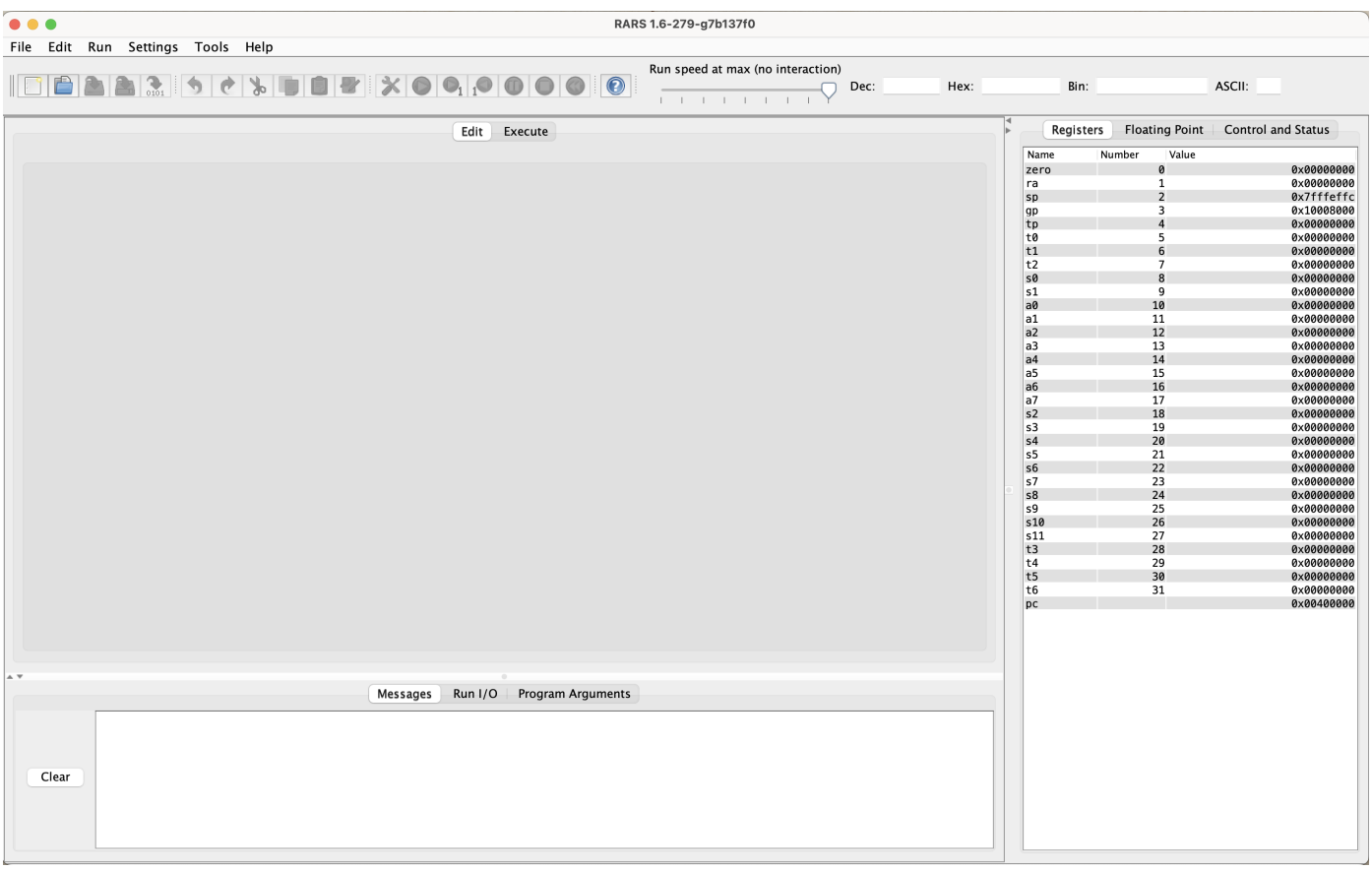
- Set up your programming environment
- Assemble and execute programs
- Debug your programs
- Where to look for RARS documentation

1 Getting started

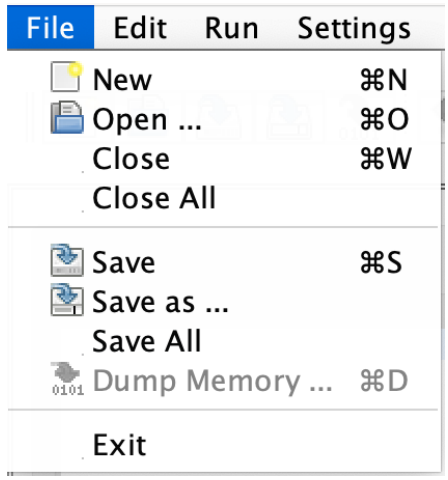
To open RARS you can either double-click the `rars.jar` file, or run the following command:

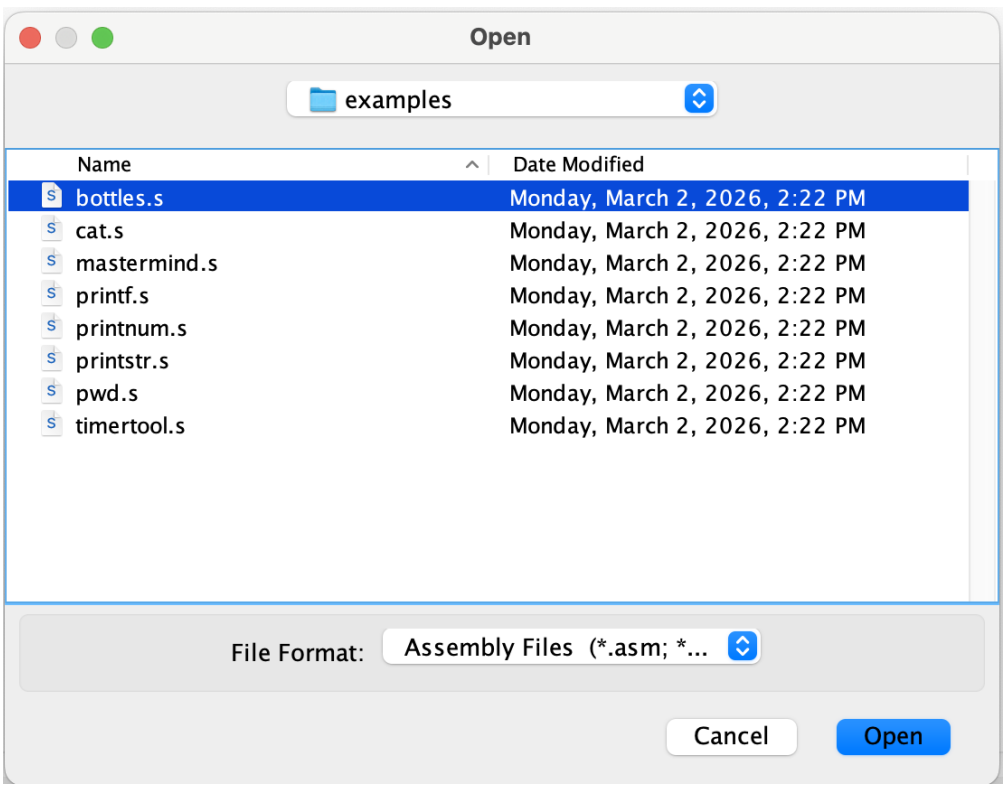
```
java -jar rars.jar
```

Both of these options will launch the Graphical User Interface (GUI) version of the RARS application.

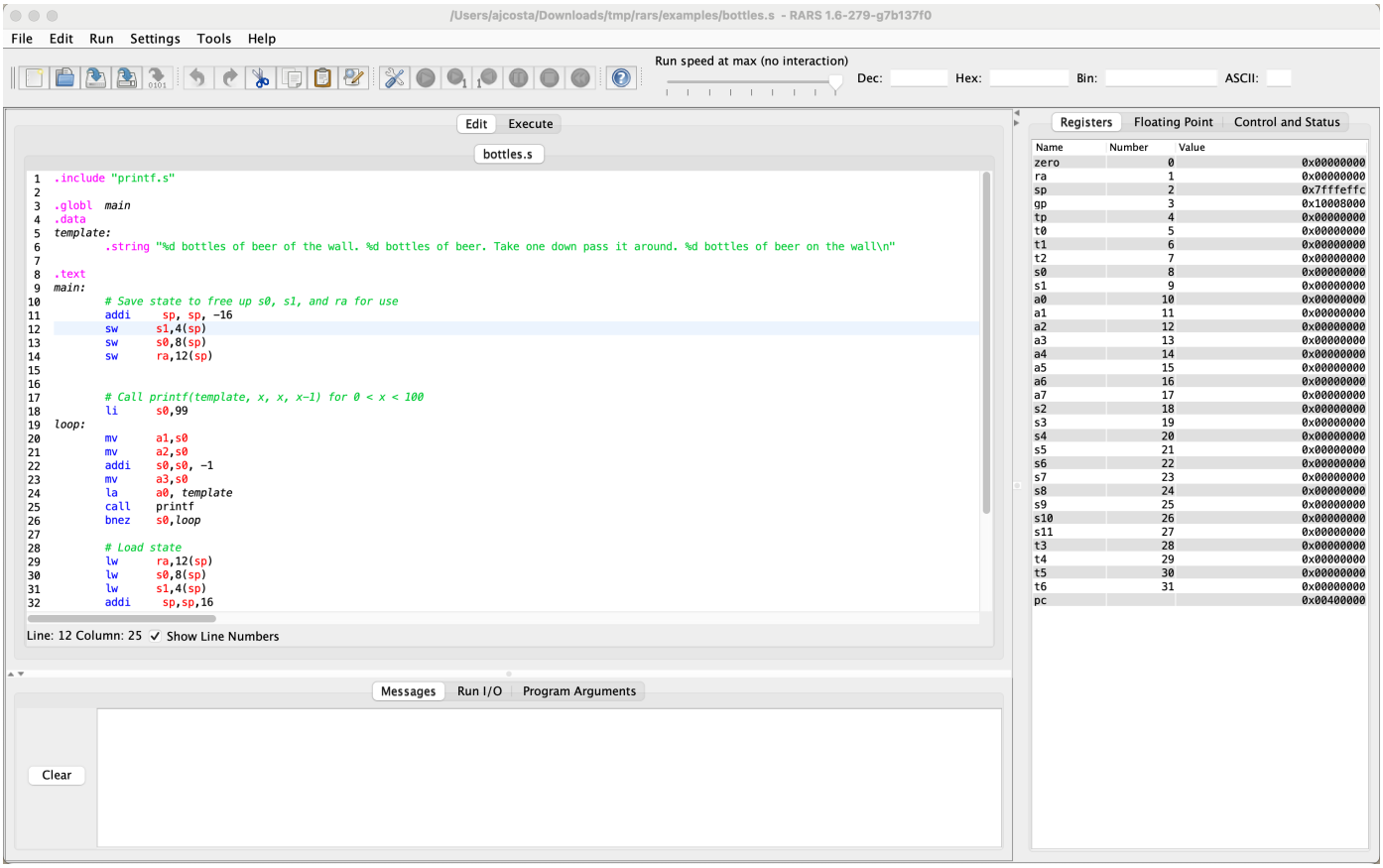


To open a source file, click on `Edit > Open ...` and then choose an assembly file. Remember that RARS assembly files should have the `.s` extension.



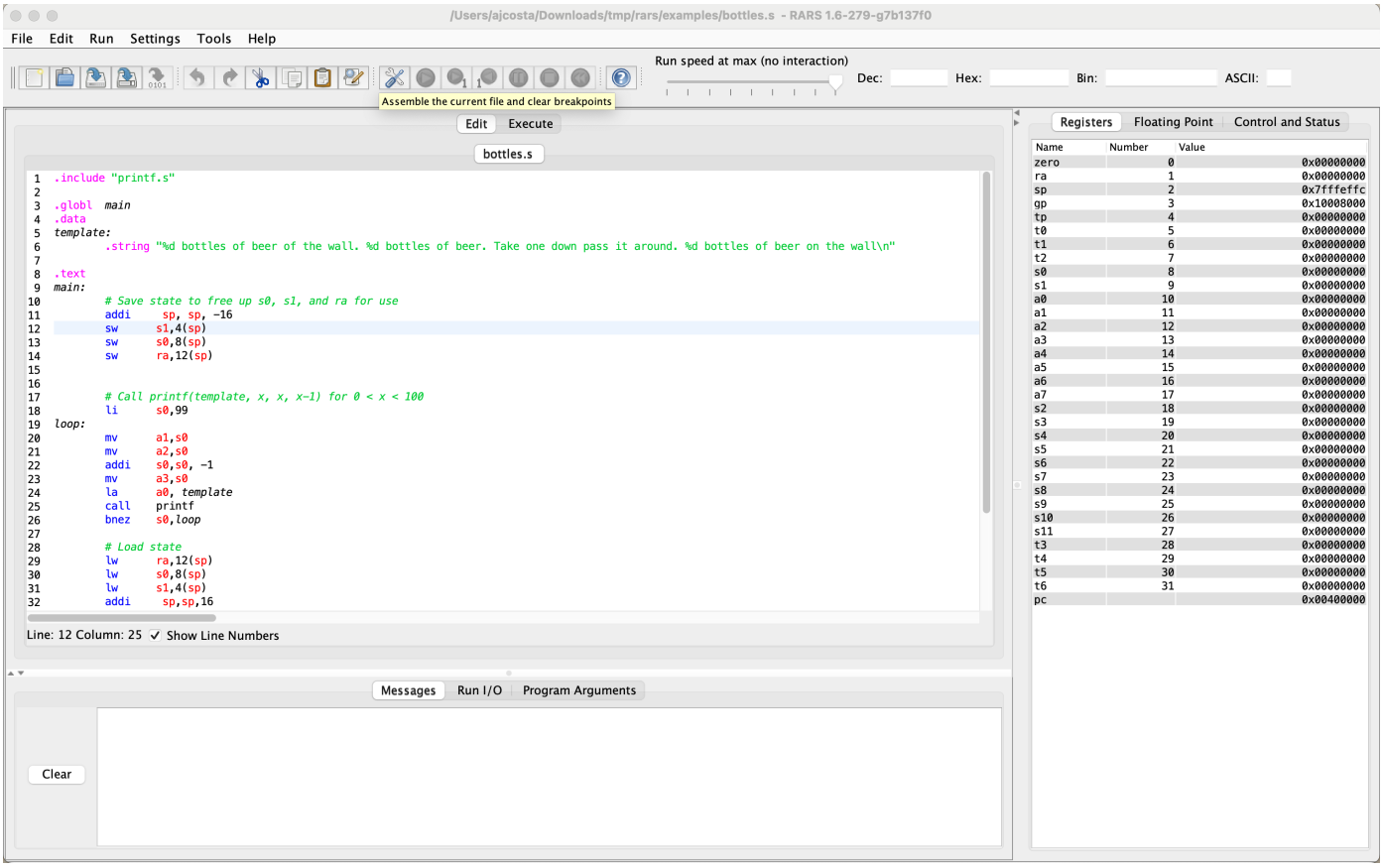


If you see a window like the following image, you should have finalized your basic environment setup.

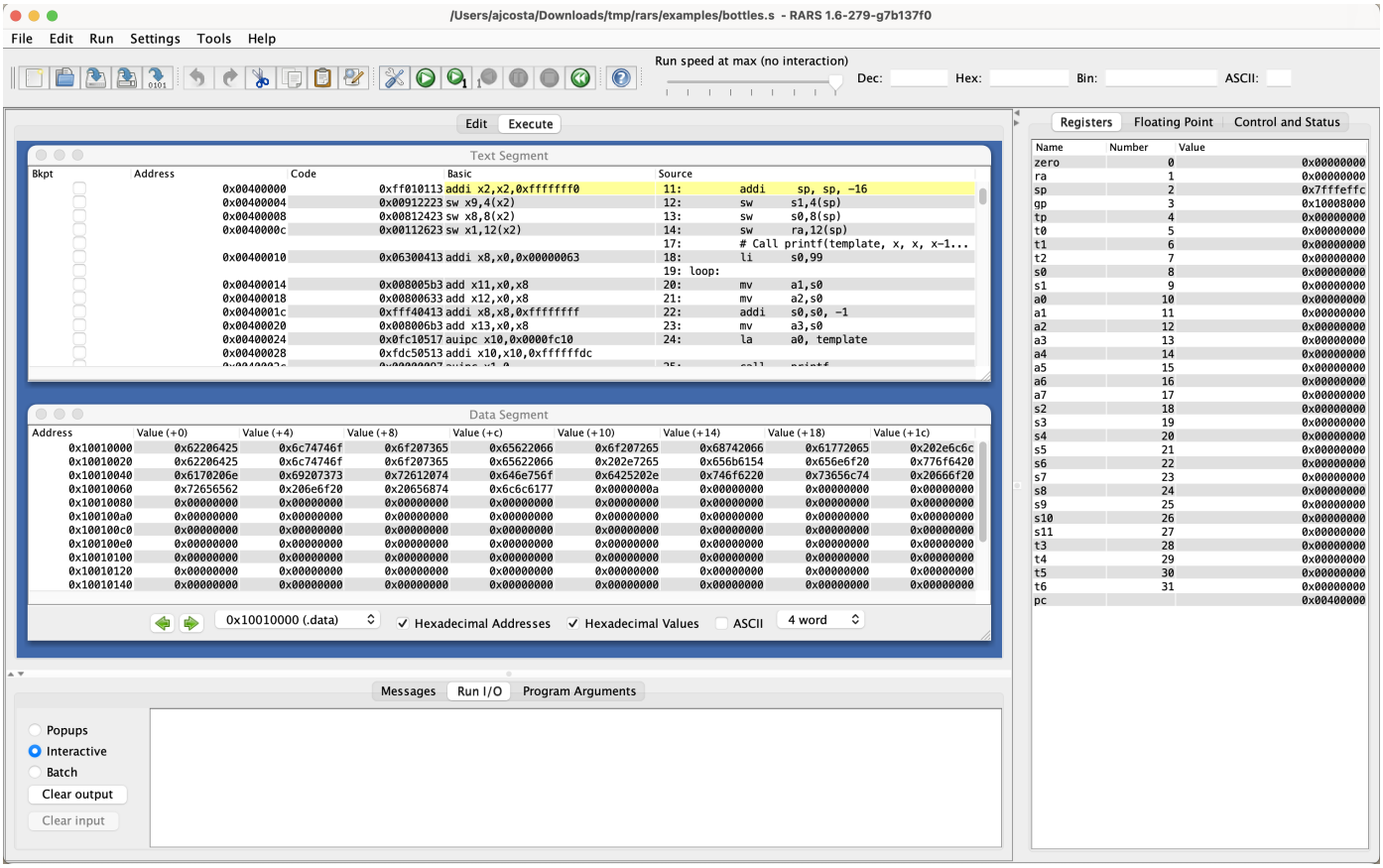

/Users/ajcosta/Downloads/tmp/rars/examples/bottles.s - RARS 1.6-279-g7b137f0
File Edit Run Settings Tools Help
Run speed at max (no interaction)
Dec:
Hex:
Bin:
ASCII:
Edit Execute
bottles.s
.include "printf.s"
.globl main
.data
template:
.string "%d bottles of beer of the wall. %d bottles of beer. Take one down pass it around. %d bottles of beer on the wall\n"
.text
main:
# Save state to free up s0, s1, and ra for use
addi sp, sp, -16
sw s1,4(sp)
sw s0,8(sp)
sw ra,12(sp)
# Call printf(template, x, x, x-1) for 0 < x < 100
li s0,99
loop:
mv a1,s0
mv a2,s0
addi s0,s0, -1
mv a3,s0
la a0, template
call printf
bnez s0,loop
# Load state
lw ra,12(sp)
lw s0,8(sp)
lw s1,4(sp)
addi sp,sp,16
Line: 12 Column: 25
Show Line Numbers
Messages Run I/O Program Arguments
Clear
Registers Floating Point Control and Status
Name Number Value
zero 0 0x00000000
ra 1 0x00000000
sp 2 0x7fffeffc
gp 3 0x10008000
tp 4 0x00000000
t0 5 0x00000000
t1 6 0x00000000
t2 7 0x00000000
s0 8 0x00000000
s1 9 0x00000000
a0 10 0x00000000
a1 11 0x00000000
a2 12 0x00000000
a3 13 0x00000000
a4 14 0x00000000
a5 15 0x00000000
a6 16 0x00000000
a7 17 0x00000000
s2 18 0x00000000
s3 19 0x00000000
s4 20 0x00000000
s5 21 0x00000000
s6 22 0x00000000
s7 23 0x00000000
s8 24 0x00000000
s9 25 0x00000000
s10 26 0x00000000
s11 27 0x00000000
t3 28 0x00000000
t4 29 0x00000000
t5 30 0x00000000
t6 31 0x00000000
pc 0x00400000

2 Running programs

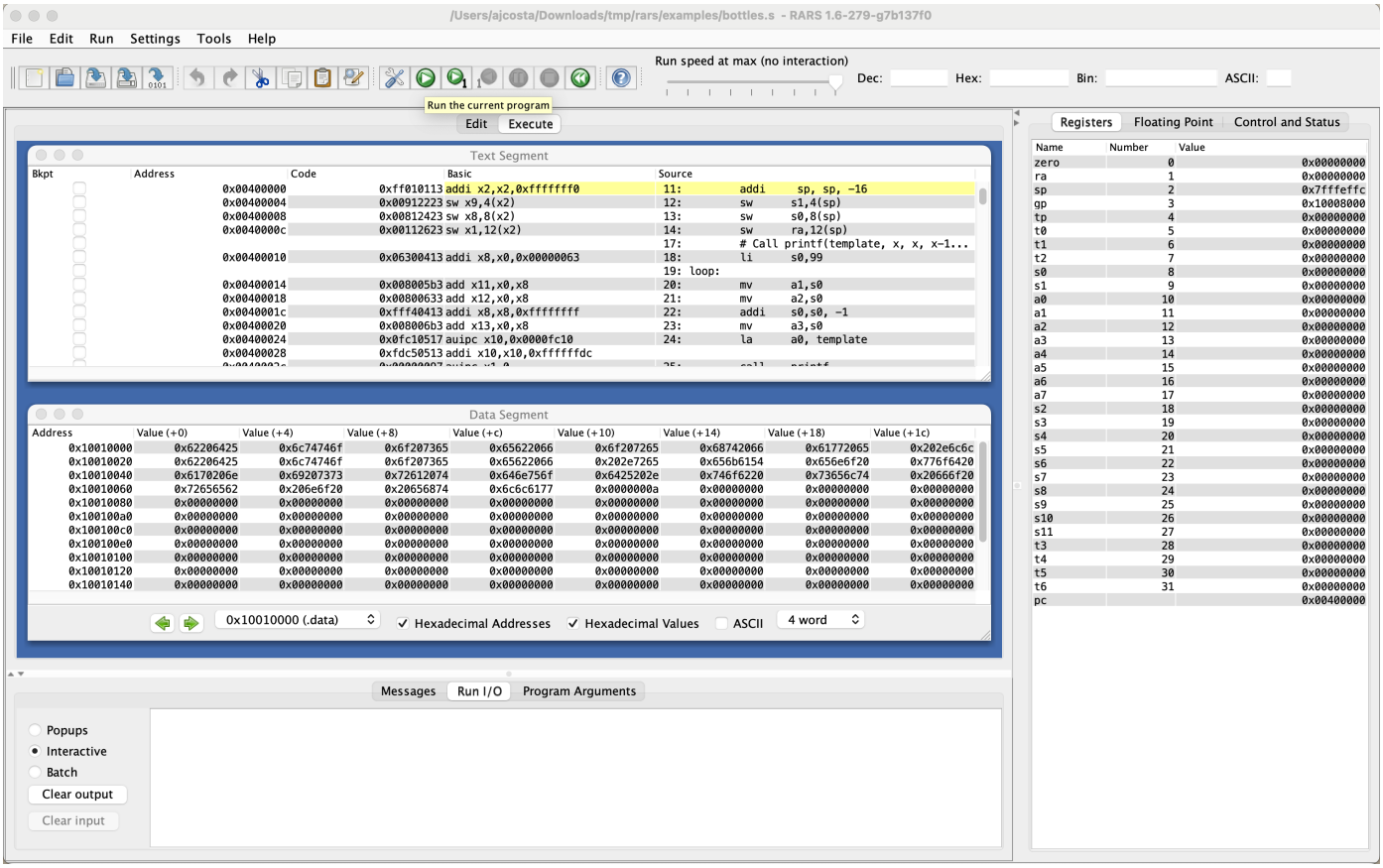
To run your program, you first need to assemble it to bytecode. To do that, click the button highlighted in the image below.



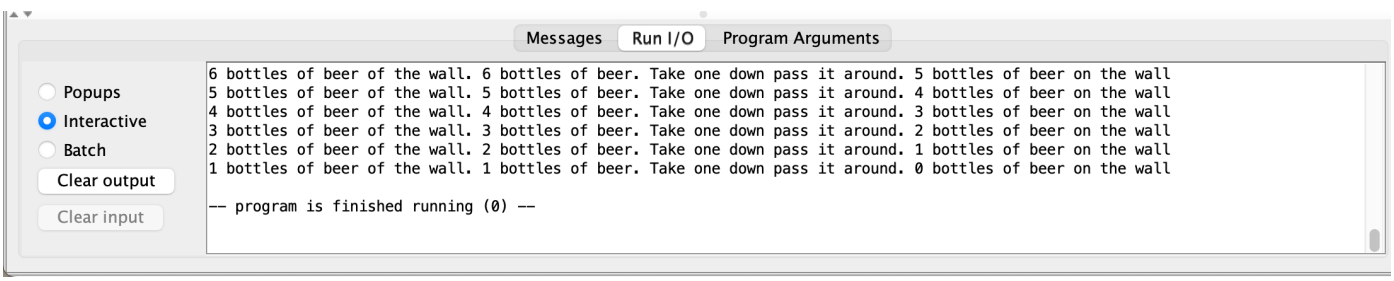
You should be automatically redirected to a window:



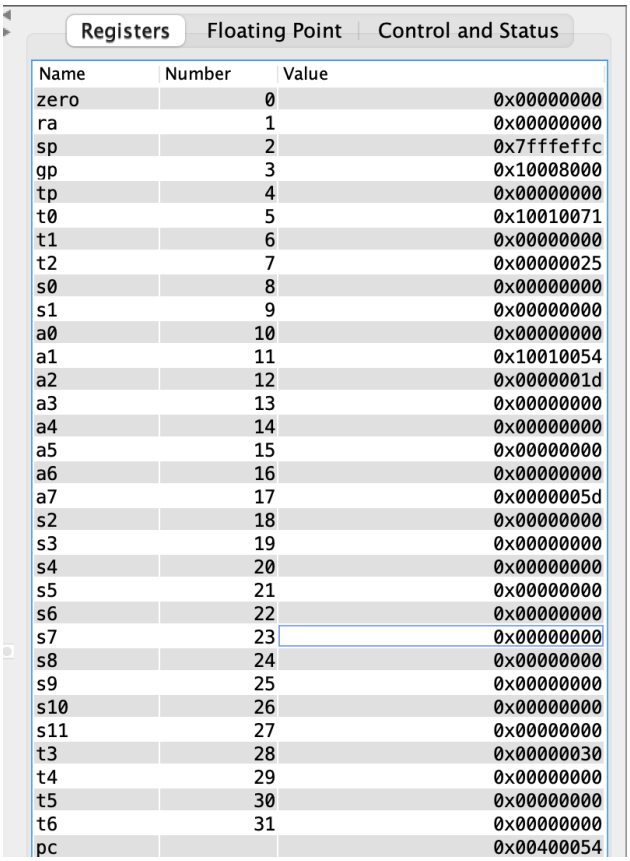
To actually run your program, click the button highlighted below. This will execute your program to completion.



You can see the (printed) output in the `Run I/O` box.



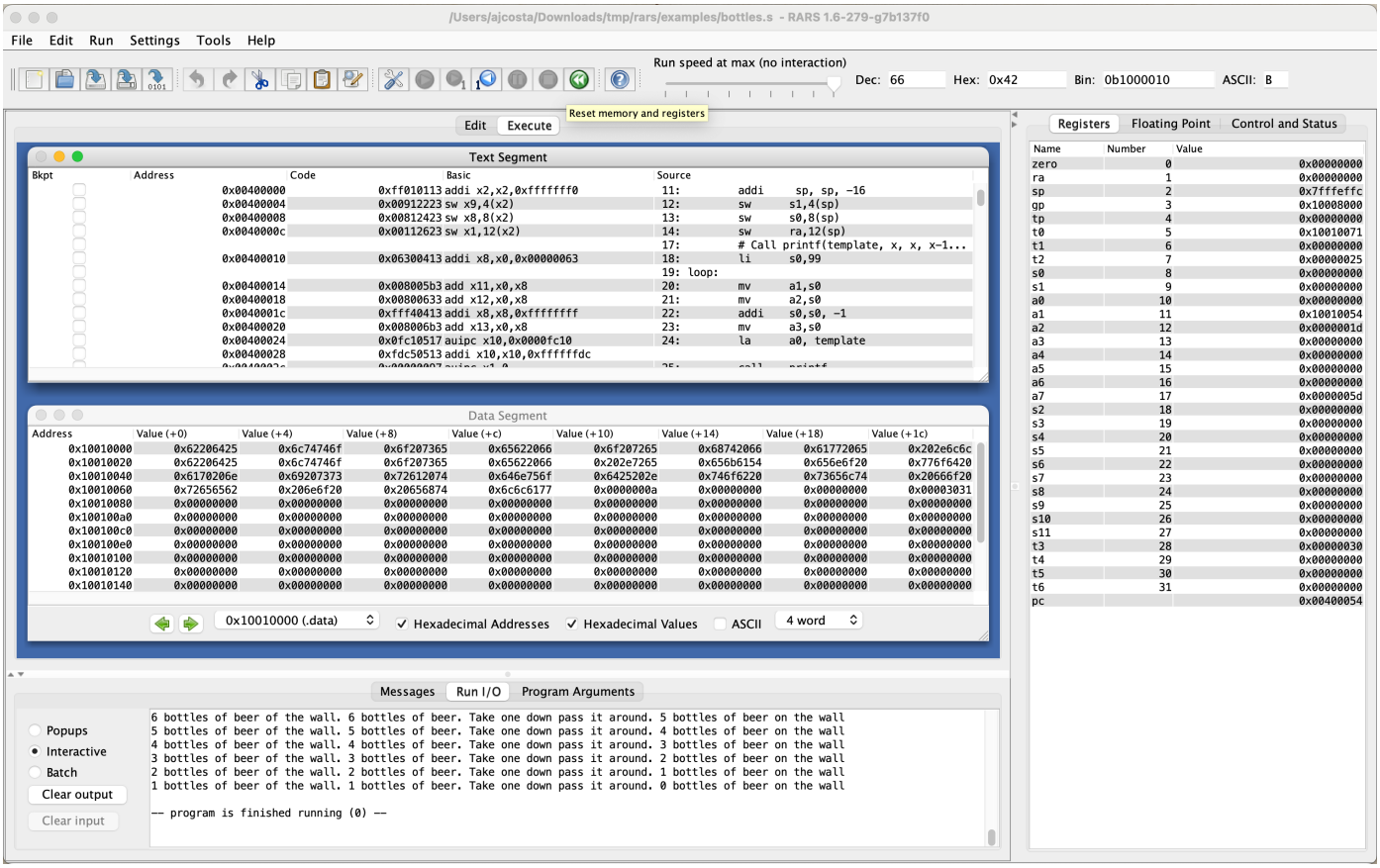
The box at the right of the window shows the values that each registers hold. By default it shows general purpose registers, but you can also look at other registers by clicking `Control and Status`/



Registers | Floating Point | Control and Status

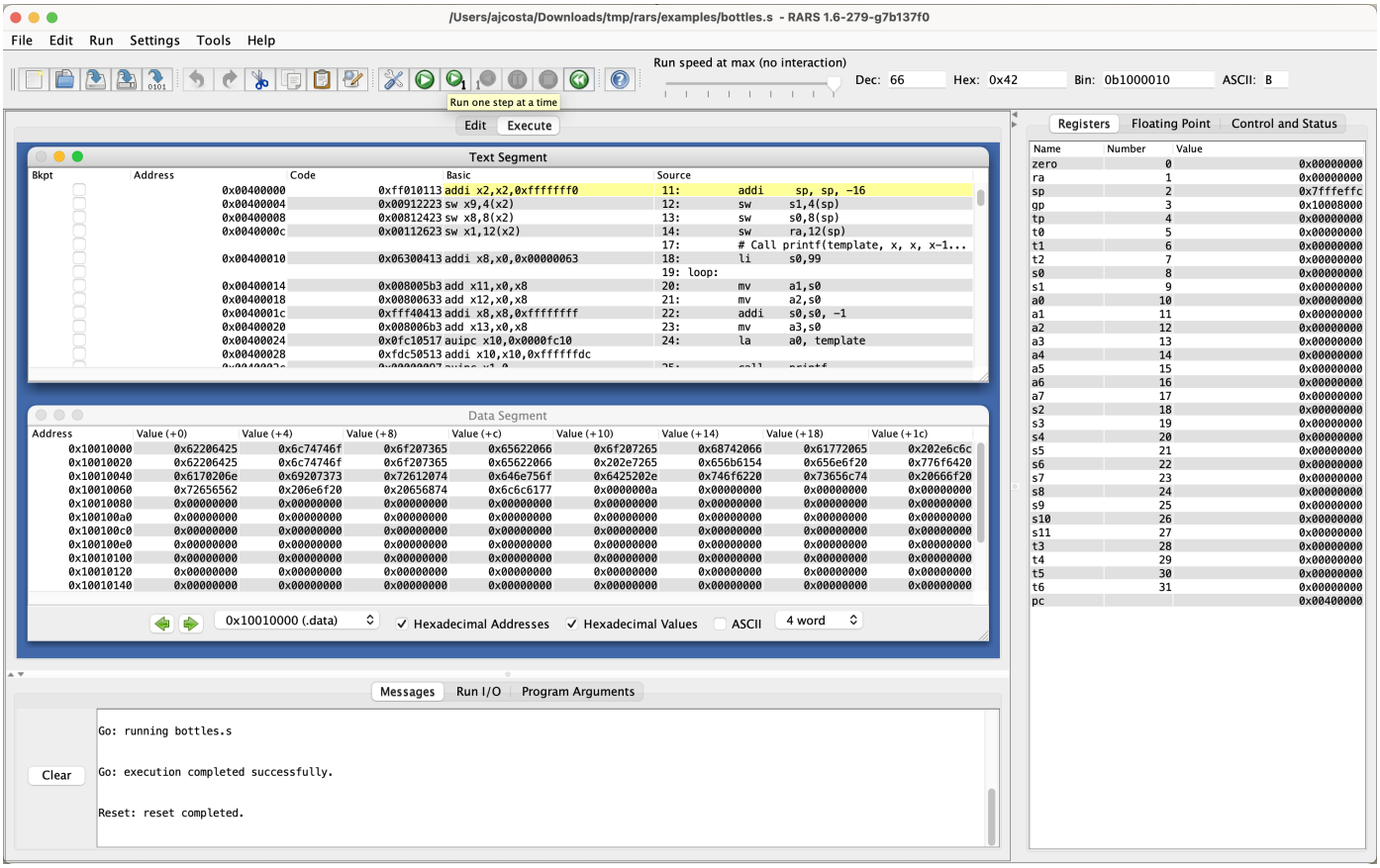
Name	Number	Value
zero	0	0x00000000
ra	1	0x00000000
sp	2	0x7fffeffc
gp	3	0x10008000
tp	4	0x00000000
t0	5	0x10010071
t1	6	0x00000000
t2	7	0x00000025
s0	8	0x00000000
s1	9	0x00000000
a0	10	0x00000000
a1	11	0x10010054
a2	12	0x0000001d
a3	13	0x00000000
a4	14	0x00000000
a5	15	0x00000000
a6	16	0x00000000
a7	17	0x0000005d
s2	18	0x00000000
s3	19	0x00000000
s4	20	0x00000000
s5	21	0x00000000
s6	22	0x00000000
s7	23	0x00000000
s8	24	0x00000000
s9	25	0x00000000
s10	26	0x00000000
s11	27	0x00000000
t3	28	0x00000030
t4	29	0x00000000
t5	30	0x00000000
t6	31	0x00000000
pc		0x00400054

To run your program again, RARS requries that you rewind each instruction that was previously executed. To rewind all instructions in one go, click the Reset memory and register button.

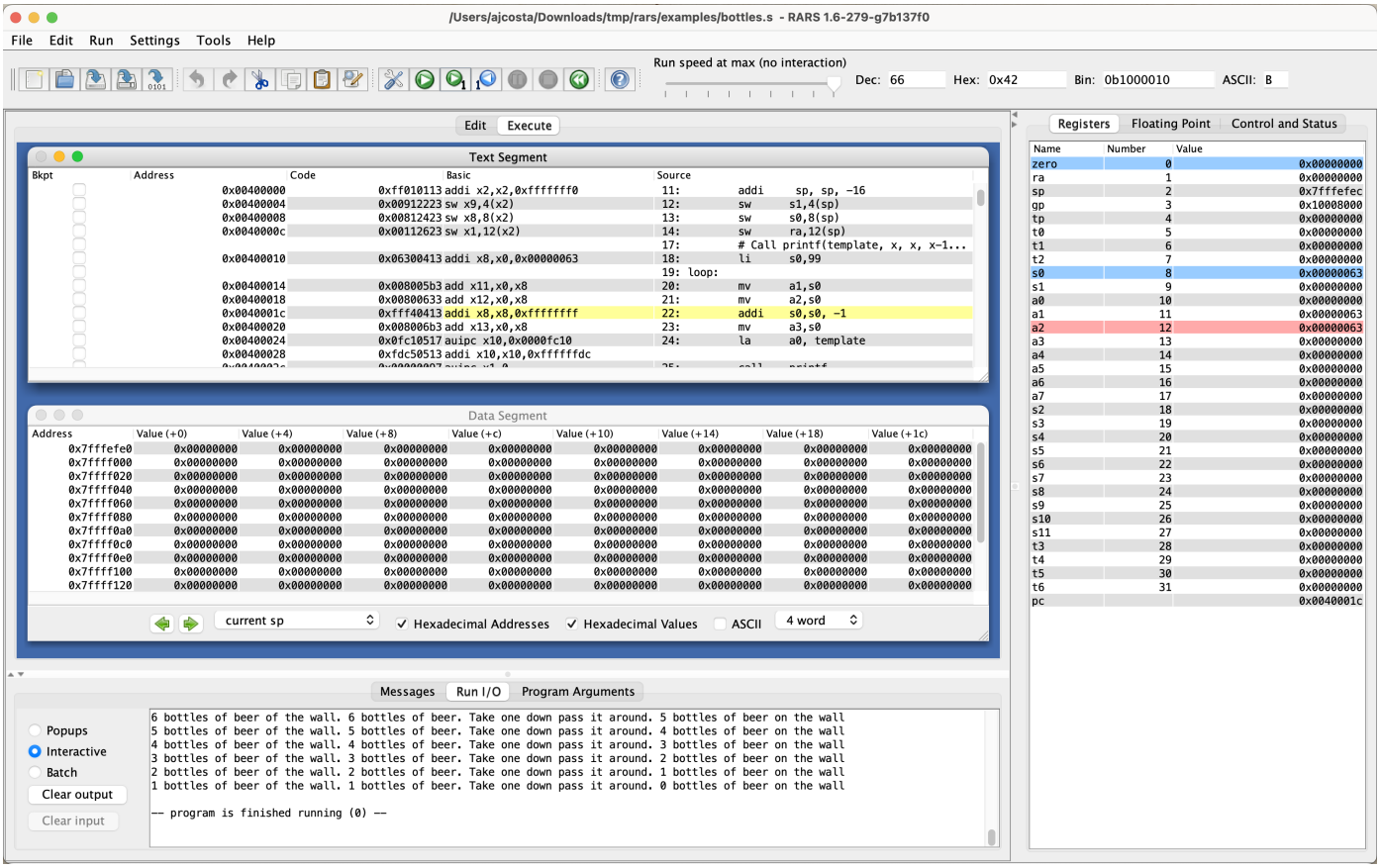


3 Debugging

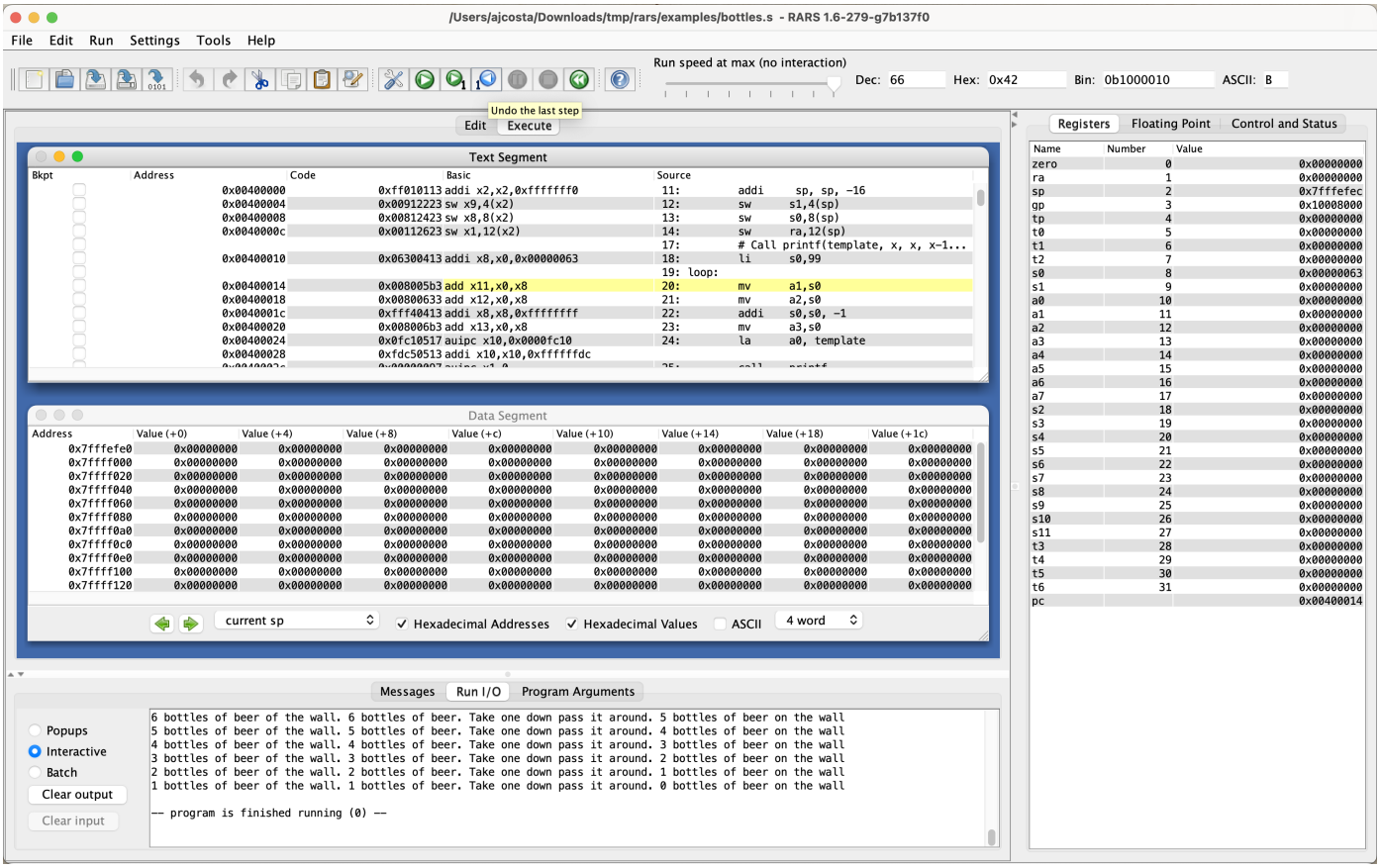
To aid in debugging your program, RARS allows you to execute one instruction at a time. To do that, click the `Run one step at a time` button.



RARS is really friendly, so it will highlight which registers were changed by each instruction! The following image shows RARS after executing the `add x12, x0, 0x8` instruction. The registers highlighted in blue show the input registers (`x0` and `x8` in this case), while the register in red show the output register (`x12`).



Like I said, RARS is **really** friendly, so it allows you to undo an instruction! Just click the `Undo the last step` button.



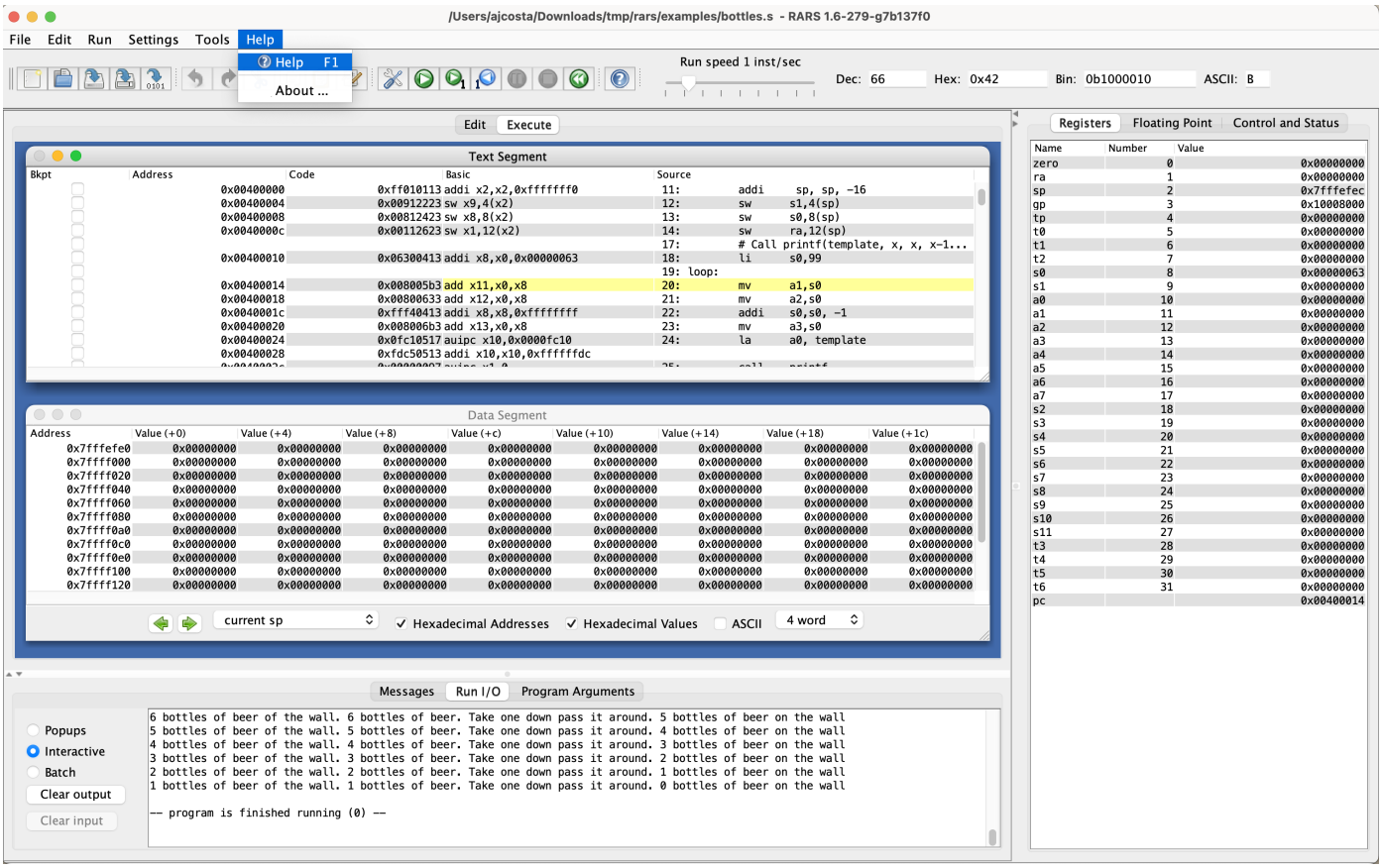
Finally, to help you look values in different representations, RARS has this box that allows you to do conversions easily.

Finally, you can also run your programs directly in the terminal! Here is an example:

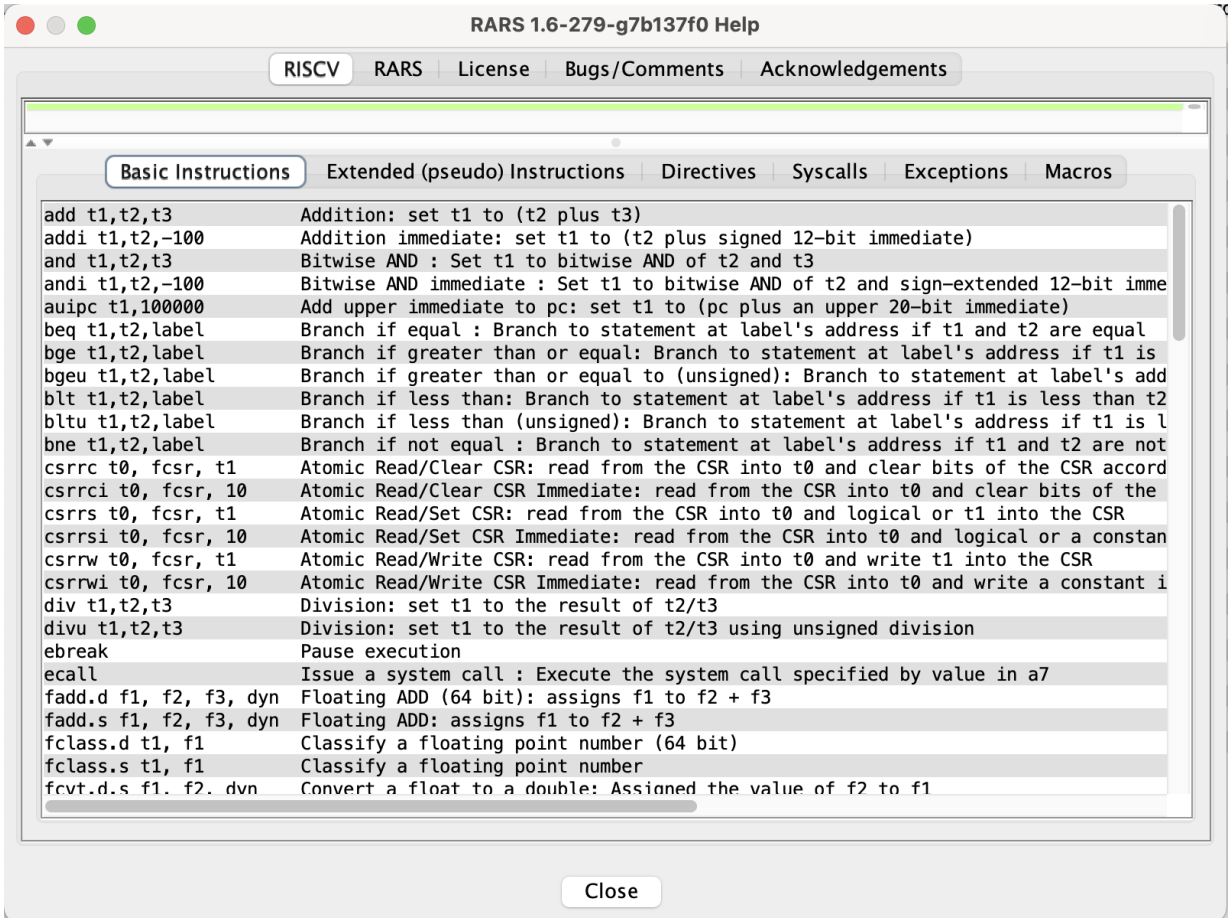
```
java -jar rars.jar ./examples/bottles.s
```

4 Documentation

If you're in need of some help, click Help!



You will reach this window, which shows you many RISC-V instructions, syscalls, and much more!



Happy assembling :)!